ResTito — Documentación Técnica

Versión: 1.1.0

Fecha: 2026-06-07

URL de producción: https://restito-production.up.railway.app

---

1. Visión General del Sistema

ResTito es un sistema POS (Point of Sale) para restaurantes que incluye:

- Gestión de mesas y comandas en salón
- Módulo de delivery/pedidos con pipeline de estados
- Pantalla KDS (Kitchen Display System) para cocina
- App móvil para repartidores con navegación GPS
- Menú online tipo carta QR para clientes con carrito y pedido por WhatsApp
- Caja y facturación
- Reportes de ventas
- Gestión de clientes, productos y usuarios
- Impresión ESC/POS via QZ Tray
- Sincronización en tiempo real vía Socket.io

Stack tecnológico

Capa	Tecnología
Backend	Node.js + Express 4
WebSockets	Socket.io 4
Base de datos	PostgreSQL (Railway)
ORM/cliente	`pg` (driver nativo)
Autenticación	JWT (jsonwebtoken) + bcryptjs
Frontend	SPA monolítica (Vanilla JS + HTML)
Impresión	QZ Tray (ESC/POS) + fallback window.print()
Deploy	Railway (branch `master`)
PWA	Service Worker + Web App Manifest

Arquitectura en ASCII

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■                  CLIENTES / DISPOSITIVOS              ■
■                                                       ■
■  /admin       /mozo       /cocina   /repartidor       ■
■  index.html   index.html  cocina    repartidor.html   ■
■  (admin rol)  (mozo rol)  .html                       ■
■                                                       ■
■  /carta                   /menu                       ■
■  carta.html               menu.html                   ■
■  (QR clientes)            (menú online)               ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                       ■ HTTP / Socket.io
■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■
■                   app.js — Backend                    ■
■                                                       ■
■  Express (API REST)   Socket.io Server                ■
■  JWT Auth Middleware  In-Memory State (db object)     ■
■  QZ Tray signing      Seed data + restoreFromPG()     ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                       ■ pg (Pool)
■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■
■          PostgreSQL Railway — tabla app_state         ■
■                                                       ■
■  Una sola fila (id=1) con columnas JSONB              ■
■  INSERT ON CONFLICT (upsert)                          ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

2. Arquitectura y Archivos

Archivos principales

Archivo	Líneas	Rol
`app.js`	1769	Backend Express + Socket.io + API REST completa
`public/index.html`	6408	SPA admin + mozo (mismo archivo, la ruta determina el rol)
`public/cocina.html`	788	Pantalla KDS cocina — sin login, solo lectura de comandas
`public/repartidor.html`	2155	App repartidor delivery — PWA con GPS
`public/carta.html`	1292	Menú online QR para clientes — dark theme, carrito, WhatsApp
`public/menu.html`	—	Alias de carta para la ruta `/menu`
`public/portal.html`	—	Portal de acceso principal (`/` y `/portal`)
`public/cliente.html`	—	Portal clientes (`/cliente`)

Rutas HTTP que sirven HTML (sin caché)

```
GET /           → portal.html
GET /portal     → portal.html
GET /admin      → index.html  (rol admin)
GET /mozo       → index.html  (rol mozo)
GET /cocina     → cocina.html
GET /repartidor→ repartidor.html
GET /carta      → carta.html
GET /menu       → menu.html
GET /cliente    → cliente.html
GET /*          → index.html  (fallback SPA)
```

Flujo de datos

```
1. Frontend carga → lee localStorage (pz_biz_cfg, pz_state, pz_user)
2. Si hay token válido → GET /api/state → restaura estado desde PostgreSQL
3. Cualquier acción del usuario → POST/PUT/PATCH a /api/...
4. Backend actualiza in-memory (db object)
5. Backend emite evento Socket.io a todos los clientes conectados
6. En acciones relevantes → POST /api/state → upsert a PostgreSQL
7. Otros clientes reciben el evento Socket.io y actualizan su UI
```

---

3. Autenticación y Roles

Mecanismo JWT

- **Secret:** `pizzeria-pro-secret-2024`
- **Expiración:** 8 horas
- **Header:** `Authorization: Bearer <token>`
- **Login:** `POST /api/auth/login` → devuelve `{ token, usuario }`
- **Logout:** `POST /api/auth/logout` (stateless, solo confirmación)

Rutas públicas (sin auth)

Estas rutas están explícitamente exentas del middleware de autenticación:

```
/api/auth/*                    — login y logout
GET /api/state                 — estado global (para restaurar SPA)
GET /api/public/menu           — menú para carta QR
/api/qz/*                      — certificados QZ Tray
GET /api/delivery/activos      — app repartidor
PUT /api/delivery/:id/estado   — app repartidor
PUT /api/delivery/:id/repartidor— app repartidor
POST /api/repartidores/login   — login específico repartidores
GET /api/repartidores          — lista de repartidores
/api/cocina/*                  — pantalla cocina (sin login)
POST /api/print                — enviar trabajo de impresión (intranet)
/api/web/*                     — pedidos desde menú online
```

Roles disponibles

Rol	Acceso	Pantalla
`admin`	Completo — todas las secciones	`/admin`
`supervisor`	Igual que admin	`/admin`
`cajero`	Caja y cobros	`/admin`
`mozo`	Solo mesas y pedidos asignados	`/mozo`
`cocinero`	Solo pantalla cocina	`/cocina`
`repartidor`	Solo app repartidor	`/repartidor`

Usuarios por defecto (seed)

Email	Contraseña	Rol
admin@restito.com	admin123	admin
supervisor@restito.com	super123	supervisor
cajero01@restito.com	cajero123	cajero
carlos@restito.com	mozo123	mozo
mozo01@restito.com	mozo123	mozo
mozo02@restito.com	mozo456	mozo
cocinero@restito.com	cocina123	cocinero
repartidor@restito.com	delivery123	repartidor

Crear nuevos usuarios

Desde el panel admin → sección Usuarios → botón "Nuevo Usuario". Se puede asignar nombre, email, contraseña y rol. Las contraseñas se hashean con bcrypt antes de guardarse en PostgreSQL.

---

4. API Endpoints Completa

Autenticación

Método	Ruta	Auth	Descripción
POST	`/api/auth/login`	No	Login con email + password, devuelve JWT
POST	`/api/auth/logout`	No	Cierre de sesión (stateless)
GET	`/api/repartidores`	No	Lista repartidores activos
POST	`/api/repartidores/login`	No	Login exclusivo para repartidores (por rol)

Mesas

Método	Ruta	Auth	Descripción
GET	`/api/mesas`	Sí	Lista todas las mesas
POST	`/api/mesas`	Sí	Crear nueva mesa
DELETE	`/api/mesas/:id`	Sí	Eliminar mesa (solo si está libre)
PATCH	`/api/mesas/:id`	Sí	Actualizar campos parciales de una mesa
POST	`/api/mesas/:id/abrir`	Sí	Abrir una mesa (asignar mozo, iniciar consumo)
POST	`/api/mesas/:id/cerrar`	Sí	Cerrar y liberar una mesa
POST	`/api/mesas/:id/pedido`	Sí	Agregar ítem al pedido de la mesa
DELETE	`/api/mesas/:id/pedido/:itemId`	Sí	Quitar ítem del pedido
GET	`/api/mesas/:id/cuenta`	Sí	Ver cuenta de la mesa (subtotal + IVA 21%)
POST	`/api/mesas/:id/transferir`	Sí	Transferir pedido a otra mesa
POST	`/api/mesas/unir`	Sí	Unir pedidos de múltiples mesas

Pedidos (Mostrador / General)

Método	Ruta	Auth	Descripción
GET	`/api/pedidos`	Sí	Lista pedidos (filtrable por `?estado=` y `?tipo=`)
GET	`/api/pedidos/:id`	Sí	Ver un pedido específico
POST	`/api/pedidos`	Sí	Crear nuevo pedido (tipo: mesa/delivery/mostrador)
PUT	`/api/pedidos/:id/estado`	Sí	Cambiar estado del pedido
POST	`/api/pedidos/:id/pagar`	Sí	Marcar pedido como pagado, genera factura, libera mesa

Delivery / Pedidos Web

Método	Ruta	Auth	Descripción
GET	`/api/delivery`	Sí	Lista todos los pedidos delivery
GET	`/api/delivery/activos`	No	Pedidos activos (para app repartidor)
POST	`/api/delivery`	Sí	Crear pedido delivery desde admin
PUT	`/api/delivery/:id/estado`	No	Cambiar estado de pedido delivery
PUT	`/api/delivery/:id/repartidor`	No	Asignar repartidor a un pedido
PUT	`/api/delivery/:id/costo-envio`	Sí	Actualizar costo de envío
POST	`/api/web/pedido`	No	Crear pedido desde menú online (carta.html)

Cocina (KDS)

Método	Ruta	Auth	Descripción
GET	`/api/cocina/comandas`	No	Lista comandas activas (filtrable por `?estado=`)
POST	`/api/cocina/comanda`	No	Crear nueva comanda (con soporte upsert para misma mesa)
PUT	`/api/cocina/comandas/:id/estado`	No	Cambiar estado de comanda (pendiente → preparacion → listo → entregado)

Productos y Catálogo

Método	Ruta	Auth	Descripción
GET	`/api/productos`	Sí	Lista productos activos
GET	`/api/productos/categorias`	Sí	Lista categorías ordenadas
POST	`/api/productos`	Sí	Crear producto
PUT	`/api/productos/:id`	Sí	Actualizar producto
DELETE	`/api/productos/:id`	Sí	Desactivar producto (soft delete)
POST	`/api/catalog`	Sí	Sincronizar productos y categorías completos

Método	Ruta	Auth	Descripción
GET	`/api/public/menu`	No	Menú público: productos + categorías + biz_cfg

Caja

Método	Ruta	Auth	Descripción
GET	`/api/caja/actual`	Sí	Estado de la caja abierta actual
POST	`/api/caja/abrir`	Sí	Abrir nueva caja con saldo inicial
POST	`/api/caja/cerrar`	Sí	Cerrar la caja actual
POST	`/api/caja/movimiento`	Sí	Registrar ingreso o egreso manual
GET	`/api/caja/resumen`	Sí	Resumen del día (ventas por método de pago)

Clientes

Método	Ruta	Auth	Descripción
GET	`/api/clientes`	Sí	Lista todos los clientes
POST	`/api/clientes`	Sí	Crear cliente
GET	`/api/clientes/:id`	Sí	Ver cliente con historial de pedidos
PUT	`/api/clientes/:id`	Sí	Actualizar cliente

Facturación

Método	Ruta	Auth	Descripción
POST	`/api/facturas`	Sí	Crear comprobante manualmente
GET	`/api/facturas/:id`	Sí	Ver comprobante con pedido asociado

Reportes

Método	Ruta	Auth	Descripción
GET	`/api/reportes/ventas`	Sí	Ventas por período (`?periodo=hoy/semana/mes`)

Método	Ruta	Auth	Descripción
GET	`/api/reportes/productos-mas-vendidos`	Sí	Top 10 productos por período
GET	`/api/reportes/dashboard`	Sí	Stats en tiempo real del dashboard

Estado global (Persistencia)

Método	Ruta	Auth	Descripción
GET	`/api/state`	No	Leer todo el estado de PostgreSQL
POST	`/api/state`	Sí	Guardar/sincronizar estado completo

Impresión

Método	Ruta	Auth	Descripción
POST	`/api/print`	No	Encolar trabajo de impresión (emit Socket.io `print:job`)
GET	`/api/print/queue`	Sí	Cola de trabajos pendientes
PATCH	`/api/print/:id`	Sí	Actualizar estado de un trabajo de impresión

QZ Tray (Firma digital)

Método	Ruta	Auth	Descripción
GET	`/api/qz/certificate`	No	Certificado PEM para QZ Tray
GET	`/api/qz/certificate.crt`	No	Descarga del certificado como archivo `.crt`
POST	`/api/qz/sign`	No	Firma SHA1 de un request QZ Tray

Llamador (Notificaciones cocina → mozo/repartidor)

Método	Ruta	Auth	Descripción
GET	`/api/llamados`	Sí	Lista llamados de las últimas 2 horas
POST	`/api/llamados/mesa`	Sí	Crear llamado para mozo de mesa

Método	Ruta	Auth	Descripción
POST	`/api/llamados/:id/recall`	Sí	Re-emitir un llamado existente
PATCH	`/api/llamados/:id`	Sí	Actualizar estado de llamado

---

5. Base de Datos

Tabla `app_state`

La base de datos tiene una sola tabla con una sola fila (id=1). Todo el estado de la aplicación vive en columnas JSONB.

```
CREATE TABLE IF NOT EXISTS app_state (
  id             INTEGER PRIMARY KEY DEFAULT 1,
  mesas          JSONB DEFAULT '[]',
  delivery       JSONB DEFAULT '[]',
  facturas       JSONB DEFAULT '[]',
  clientes       JSONB DEFAULT '[]',
  usuarios       JSONB DEFAULT '[]',
  productos      JSONB DEFAULT '[]',
  mozo_historial JSONB DEFAULT '[]',
  caja_abierta   BOOLEAN DEFAULT TRUE,
  caja_inicial   INTEGER DEFAULT 5000,
  caja_moves     JSONB DEFAULT '[]',
  caja_cierres   JSONB DEFAULT '[]',
  categorias     JSONB DEFAULT '[]',
  biz_cfg        JSONB DEFAULT '{}',
  updated_at     TIMESTAMPTZ DEFAULT NOW()
);
```

Estrategia de escritura: INSERT ON CONFLICT DO UPDATE (upsert sobre id=1).

Restauración al arranque: Al iniciar, `restoreStateFromPG()` lee las columnas y sobreescribe el in-memory `db` object solo si el array tiene elementos. El seed data permanece como fallback si no hay datos en PG.

Estructura de entidades

Mesa

```
{
  "id": "uuid",
  "numero": 3,
  "zona": "salon",
  "capacidad": 4,
  "estado": "libre | ocupada | cuenta",
  "mozoid": "uuid | null",
  "mozo": "nombre | null",
  "apertura": "ISO timestamp | null",
  "tiempo": "00:45 | null",
  "consumo": 4200,
  "pedido": [],
  "pedidos": [
    {
      "id": "uuid",
      "productoId": "uuid",
      "nombre": "Pizza Muzzarella",
      "variante": "grande",
      "cantidad": 2,
      "precio": 2100,
      "extras": [],
      "observacion": ""
    }
  ]
}
```

Producto

```
{
  "id": "uuid",
  "codigo": "PIZ001",
  "nombre": "Muzzarella",
  "descripcion": "Clásica pizza de muzzarella",
  "categoria": "uuid-categoria",
  "precio": 1200,
  "precioMediano": 1600,
  "precioGrande": 2100,
  "stock": 100,
  "stockMinimo": 5,
  "imagen": "",
  "activo": true,
  "extras": [
    { "id": "e1", "nombre": "Aceitunas", "precio": 150 }
  ]
}
```

Los campos `precioMediano` y `precioGrande` son `null` para productos de precio único.

En `carta.html`, los productos con múltiples precios usan la propiedad `precios` (objeto clave-valor de tamaños/presentaciones).

Categoría

```
{
  "id": "uuid",
  "nombre": "Pizzas",
  "icono": "■",
  "orden": 1
}
```

Pedido Delivery (in-memory `db.delivery` y columna PG)

```
{
  "id": 1717000000000,
  "numero": "W-0000",
  "estado": "nuevo | en_cocina | listo | en_camino | entregado | cancelado",
  "cliente": {
    "nombre": "Juan García",
    "telefono": "11-4444-5555"
  },
  "direccion": "Av. Corrientes 1234",
  "items": [
    { "nombre": "Pizza Muzzarella", "qty": 1, "precio": 2100, "size": "grande", "nota": "" }
  ],
  "total": 2100,
  "metodo_pago": "Transferencia",
  "nota": "",
  "hora": "14:30",
  "origen": "web | admin",
  "fecha": "2026-06-07",
  "costo_envio": 500,
  "modo_envio": "delivery | retiro",
  "distancia_km": 3.2,
  "lat_cliente": -34.6037,
  "lon_cliente": -58.3816,
  "repartidorId": "uuid | null",
  "repartidorNombre": "Repartidor 01 | null"
}
```

Comanda (in-memory solamente)

```
{
  "id": "uuid",
  "numero": 42,
  "tipo": "mesa | delivery",
  "mesa": 3,
  "mesaId": "uuid | null",
  "mozo": "Carlos Mozo",
  "cliente": "Juan García | null",
  "items": [
    {
      "nombre": "Pizza Muzzarella",
      "qty": 2,
      "variante": "grande",
      "nota": ""
    }
  ],
  "estado": "pendiente | preparacion | listo | entregado",
  "createdAt": "ISO timestamp"
}
```

Factura

```
{
  "id": "uuid",
  "numero": "F-000001",
  "tipo": "B",
  "pedidoId": "uuid | null",
  "total": 5000,
  "subtotal": 4132.23,
  "iva": 867.77,
  "metodoPago": "efectivo",
  "createdAt": "ISO timestamp"
}
```

Cliente

```
{
  "id": "uuid",
  "nombre": "Juan García",
  "email": "juan@ejemplo.com",
  "telefono": "11-4444-5555",
  "direcciones": [
    {
      "id": "uuid",
      "calle": "Av. Corrientes 1234",
      "barrio": "Centro",
      "referencia": "Piso 3 dpto B"
    }
  ],
  "historial": [],
  "createdAt": "ISO timestamp"
}
```

Usuario

```
{
  "id": "uuid",
  "nombre": "Carlos Mozo",
  "email": "carlos@restito.com",
  "password": "$2b$10$...",
  "rol": "admin | supervisor | cajero | mozo | cocinero | repartidor",
  "activo": true,
  "telefono": "",
  "createdAt": "ISO timestamp"
}
```

Configuración del negocio (`biz_cfg`)

```
{
  "nombre": "Pizzería ResTito",
  "dir": "Av. Corrientes 1234",
  "tel": "011-4444-5555",
  "wsp": "5491144445555",
  "alias": "RESTITO.MP",
  "cbu": "0000003100012345678901",
  "logo": "...",
  "recibo": "¡Gracias por su visita!",
  "latitud": "-34.6037",
  "longitud": "-58.3816",
  "radioDelivery": 10,
  "costoEnvio": 500,
  "precioPorKm": 150,
  "mediosPago": [
    { "id": "efectivo", "nombre": "Efectivo", "icono": "■", "recargo": 0 },
    { "id": "tarjeta", "nombre": "Tarjeta", "icono": "■", "recargo": 5 },
    { "id": "transferencia", "nombre": "Transferencia", "icono": "■", "recargo": 0 }
  ],
  "propinaPct": 10
}
```

LocalStorage (Frontend)

Clave	Contenido
`pz_biz_cfg`	Configuración del negocio (biz_cfg)
`pz_state`	Estado completo (mesas, productos, delivery, facturas, clientes, caja)
`pz_user`	Usuario logueado actual (id, nombre, email, rol, token)
`pz_print_cfg`	Configuración de impresora (paperWidth: 58mm/80mm, qzPrinter)
`rep_driver`	Sesión del repartidor (restauración automática al recargar)
`restito_rep_history`	Historial de entregas del repartidor (local, hasta 300 registros)
`restito_rep_comments`	Comentarios del repartidor por pedido

---

6. Sistema de Socket.io — Eventos en Tiempo Real

Eventos que emite el servidor (`io.emit` / `socket.emit`)

Evento	Payload	Cuándo se emite
`mesa:update`	objeto mesa	Al abrir/cerrar/modificar/agregar ítem/transferir/unir mesas
`mesa:deleted`	`{ id }`	Al eliminar una mesa
`pedido:nuevo`	objeto pedido	Al crear pedido

Evento	Payload	Cuándo se emite
`pedido:update`	objeto pedido	Al cambiar estado o pagar
`delivery:update`	objeto delivery	Al crear/cambiar estado/asignar repartidor
`delivery:init`	array de activos	Al unirse a la sala `delivery`
`comanda:nueva`	objeto comanda	Al crear comanda
`comanda:update`	`{ id, estado }`	Al cambiar estado de comanda
`comanda:replace`	objeto comanda	Al hacer upsert de comanda (actualizar ítems misma mesa)
`cocina:init`	array de comandas	Al unirse a sala `cocina`
`cocina:update`	objeto comanda	Al cambiar estado de comanda desde cocina
`caja:update`	objeto caja	Al abrir/cerrar caja o registrar movimiento
`print:job`	objeto job	Al encolar trabajo de impresión
`print:queue:update`	array de jobs	Al cambiar cola de impresión
`dashboard:stats`	objeto stats	Cada 10 segundos + en eventos relevantes
`llamado:mesa`	objeto llamado	Cuando comanda de mesa queda lista
`llamado:delivery`	objeto llamado	Cuando delivery pasa a estado `listo`
`llamado:update`	objeto llamado	Al reconocer/re-llamar un llamado
`state:changed`	`{ updated_at }`	Al guardar estado en PostgreSQL

Eventos que escucha el servidor (desde clientes)

Evento	Descripción
`join:room`	El cliente se une a una sala (`cocina`, `dashboard`, `delivery`)
`client:mesa:update`	El cliente sincroniza una mesa, el servidor la re-emite a otros
`delivery:status`	Repartidor actualiza estado de delivery
`comanda:update`	Cocina actualiza estado de comanda
`delivery:broadcast`	Admin difunde un pedido delivery completo
`llamado:ack`	Mozo/repartidor reconoce un llamado

Dashboard Stats — estructura del payload

```
{
  "mesasOcupadas": 3,
  "mesasLibres": 6,
  "pedidosActivos": 5,
  "deliveryActivos": 2,
  "ventaHoy": 45000,
  "saldoCaja": 38000,
  "comandasPendientes": 4,
  "timestamp": "ISO timestamp"
}
```

---

7. Sistema de Impresión ESC/POS

QZ Tray

QZ Tray es una aplicación de escritorio que debe estar instalada en la PC que tiene conectada la impresora. El sistema carga `qz-tray.js` desde el servidor y se autentica con un certificado firmado (SHA1).

Archivos de certificado:

- `qz-cert.pem` — certificado público
- `qz-key.pem` — clave privada para firma

Si los archivos no existen, QZ Tray opera en modo anónimo con funcionalidad limitada.

Flujo de impresión

```
1. Frontend llama a POST /api/print con { type, html, mesaNumero, items }
2. Backend emite print:job via Socket.io
3. Dispositivo admin (con QZ Tray activo) recibe el evento
4. Si QZ Tray disponible: imprime via ESC/POS directo
5. Fallback: window.open() + window.print() en ventana nueva
```

Funciones ESC/POS en `index.html`

Función	Propósito
`_ep(mesa, lines, header, footer)`	Builder base — construye el documento ESC/POS
`epComanda(mesa, items)`	Ticket de cocina (comanda de mesa)
`epCuenta(mesa, items)`	Ticket de cuenta de mesa (para el cliente)
`epComprobanteX(f)`	Comprobante X — cierre de mesa con datos cliente editables
`epComandaDelivery(o)`	Ticket de comanda para pedido delivery

Configuración de papel

Ancho	Columnas	Selección
58mm	32 columnas	Radio button en Config
80mm	48 columnas	Radio button en Config (defecto)

La configuración se guarda en `localStorage` bajo `pz_print_cfg`.

Formato de ítems en tickets

```
3x Coca Cola 500ml
$600 x un.    subt. $1800
```

Implementado calculando el gap entre columna izquierda (precio unitario) y columna derecha (subtotal) según el ancho de papel configurado.

---

8. Módulo de Catálogo (Productos)

Tipos de precio

Tipo	Descripción	Campos usados
Precio único	Un solo precio	`precio`
Por tamaño (Chica/Mediana/Grande)	2 o 3 precios	`precio`, `precioMediano`, `precioGrande`
Por presentación	Filas dinámicas (ej: Entero/Mitad)	`precios: { "Entero": 2000, "Mitad": 1200 }`

En el menú online (`carta.html`), los productos con `precios` (objeto) muestran chips de precio por variante. Los de `precio` único muestran un solo valor.

Categorías

Cada categoría tiene:

- `id` (UUID)
- `nombre`
- `icono` (emoji)
- `orden` (número entero para ordenamiento)

El campo `orden` se usa en `GET /api/productos/categorias` para devolver las categorías en el orden correcto.

---

9. Deploy y Configuración

Variables de entorno Railway

Variable	Descripción
`DATABASE_URL`	Connection string PostgreSQL (inyectada automáticamente por Railway)
`PORT`	Puerto del servidor (inyectado automáticamente, defecto 3000)

Iniciar localmente

```
# Con PostgreSQL local
DATABASE_URL="postgresql://user:pass@localhost:5432/restito" node app.js

# Sin PostgreSQL (modo in-memory, datos se pierden al reiniciar)
node app.js
```

Git Flow obligatorio

Siempre hacer push a los 3 branches antes de deployar:

```
git push origin master-sync
git push origin master-sync:master
git push origin master-sync:claude/portal-replica-multiagent-H53V3
```

Deploy a Railway con SHA exacto

```
SHA=$(git rev-parse HEAD)
curl -s -X POST https://backboard.railway.app/graphql/v2 \
  -H "Authorization: Bearer 55ea6497-1857-4e12-a4ab-a31565de4d0c" \
  -H "Content-Type: application/json" \
  -d "{\"query\":\"mutation { serviceInstanceDeploy(serviceId: \\\"7d9be2a3-4f7c-4fdd-b93c-21f6d67
```

Importante: Usar siempre `serviceInstanceDeploy` con `commitSha` explícito. NO usar `serviceInstanceDeployV2` ya que usa un commit cacheado.

IDs Railway

Recurso	ID
Proyecto	`ea348246-a9fb-4251-89a3-1f469cb26fbc`
Servicio app	`7d9be2a3-4f7c-4fdd-b93c-21f6d6796aa5`
Servicio DB	`1c3c173c-a230-4e8e-8f5a-f03bb96af1ff`
Environment	`3b68f44b-d69f-4933-b8c1-063b68d23b3f`
API Token	`55ea6497-1857-4e12-a4ab-a31565de4d0c`

Verificar estado del deploy

```
# Ver deployments recientes
curl -s -X POST https://backboard.railway.app/graphql/v2 \
  -H "Authorization: Bearer 55ea6497-1857-4e12-a4ab-a31565de4d0c" \
  -H "Content-Type: application/json" \
  -d '{"query":"{ deployments(input:{serviceId:\"7d9be2a3-4f7c-4fdd-b93c-21f6d6796aa5\",environmen
```

---

10. Notas de Implementación

Sincronización de estado dual (in-memory + PostgreSQL)

El backend mantiene un objeto `db` en memoria (la fuente de verdad para la sesión actual) y sincroniza con PostgreSQL al:

1. Arranque del servidor (`restoreStateFromPG`)
2. Cada vez que el frontend llama `POST /api/state` (después de cualquier cambio)
3. Directamente en rutas críticas de delivery (para que el repartidor siempre vea datos frescos)

Los pedidos web (`POST /api/web/pedido`) se escriben directamente a PostgreSQL además del in-memory, para garantizar persistencia incluso si el server se reinicia entre el pedido y cuando el admin lo revisa.

Limpieza automática

- **Trabajos de impresión:** Expiración 1 hora, limpieza cada 5 minutos
- **Llamados:** Limpieza de llamados con más de 2 horas, cada 30 minutos
- **Comandas:** Cap de 500 comandas en memoria (las más viejas se descartan)

PWA (Progressive Web App)

Las pantallas mozo, admin y repartidor tienen soporte PWA con:

- `manifest.json` específico por rol
- Service Worker (`/sw.js`)
- Iconos SVG por rol
- `apple-mobile-web-app-capable` para instalación en iOS